Why Is Redis a Distributed Swiss Army Knif

#50: Break Into Redis Use Cases (5 Minutes)

NEO KIM
JUN 20, 2024

Note: This isn't a sponsored post. I wrote it for someone who has never used Redis. will find references at the bottom of this page if you want to go deeper.

- *Share this post & I'll send you some rewards for the referrals.*

Why it matters: Redis is often used to build high-performance apps and solve v problems without reinventing the wheel.

Between the lines: Redis changed their licensing recently. Yet there are many fo

It's hackathon day at a tech company named Hooli.

And 2 software engineers want to create a fancy little app.

Yet they were experienced only with a few tools - including Redis.

So they decide to reuse their skills in every possible way and build a prototype.

Redis Use Cases

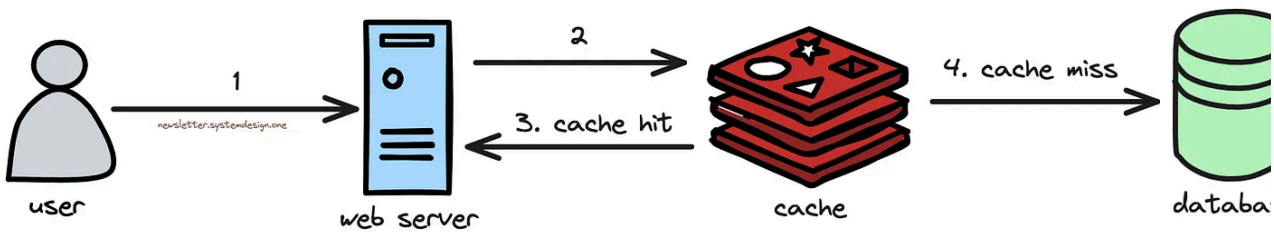
Here's how they built the app:

1. Caching:

They installed a web server to handle API requests.

Yet most likely the same data will often get queried from the database. So they s a Redis cache in front of the database.

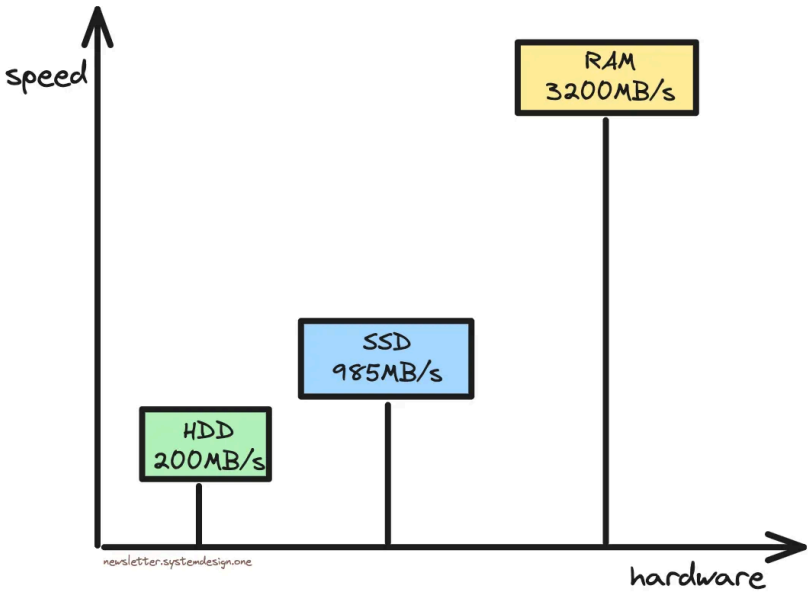
A **cache** is a fast, in-memory data store.



Caching Database Response

How it works:

- The database response is cached in the Redis hash data structure
- The database query gets hashed and is stored as the cache key
- The cache gets queried on every request to check if the data is present
- Data is in cache: response gets served by the cache - **cache hit**
- Data isn't in the cache: query hits the database and then fills the cache - **cac miss**



Speed Comparison

It offers:

- Faster response - 100x
- Reduced database load

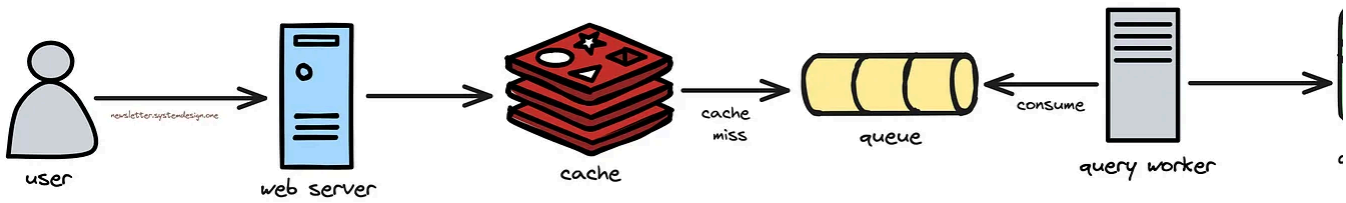
2. Queueing:

They added extra servers to query and process data - named **query workers**.

Yet some queries took longer to finish.

So they set up Redis streams to queue requests. It allowed them to process requ asynchronously.

A Redis **stream** is a data structure that acts like an append-only log. This means entry is immutable.



Queueing Requests for Asynchronous Processing

How it works:

1. The request gets forwarded on a cache miss
2. The request gets wrapped in a message and gets assigned a unique identifie
3. The message gets added to the queue
4. The query worker consumes the message if it has free capacity
5. The query worker interacts with the database

It let them:

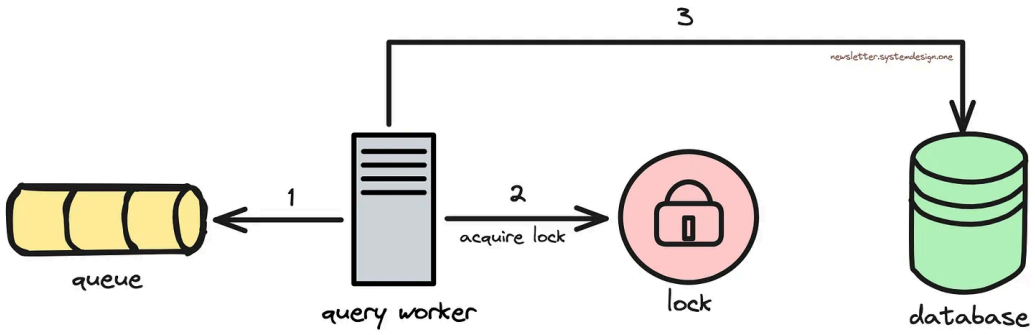
- Buffer requests for asynchronous processing
- Decouple serving and processing of requests
- Auto-scale query workers based on load

3. Locking:

Many expensive queries at once will overload the database.

So they installed a distributed lock using Redis.

A **distributed lock** orchestrates access to a shared resource from many clients.



Acquiring a Lock Before Querying the Database

How it works:

- The query worker acquires a lock before talking to the database
- An expiry time is set for the lock
- The number of query workers that access the database at once is limited. Wh others must wait

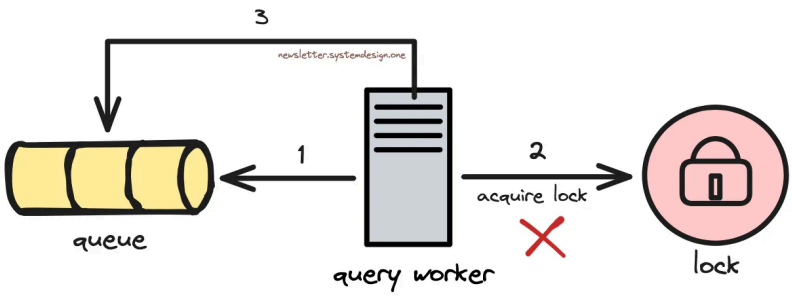
It offers:

- Resilience by avoiding many queries at once
- Avoid noisy neighbor problem

4. Throttling:

There's a risk of lock acquisition failure. It will overload the database.

So they use Redis streams to throttle messages.



Re-Inserting Messages to the Queue for Throttling

How it works:

- The query worker fails to acquire a lock, so the message gets added back to queue
- Also a delay gets assigned to the message before inserting it
- And an extra delay gets added each time the same message is re-inserted in queue - **exponential backoff**

It provides:

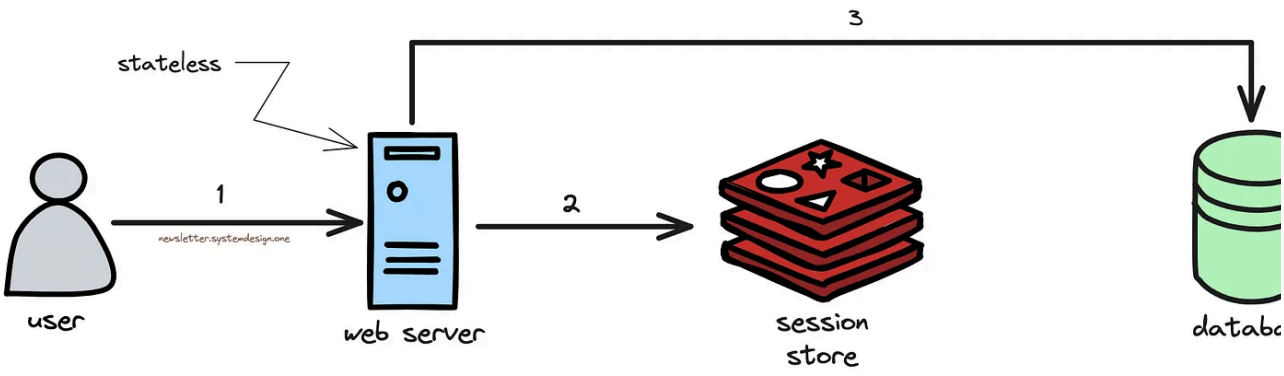
- Congestion management by controlling the number of parallel database req

5. Session Store:

The web server stores the user's data and preferences.

Yet it's hard to scale a *stateful* web server.

So they installed a separate session store using Redis.



Storing Session Data in a Separate Store

How it works:

- Session data is stored in the Redis hash data structure
- An expiry time is set for each user's data
- The expiry time gets renewed whenever the user requests something

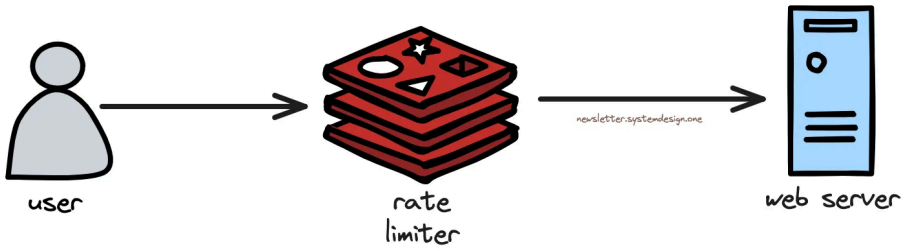
It let them:

- Scale *stateless* web servers easily
- Handle traffic spikes

6. Rate Limiting:

Many API requests in a short period will overload the web server.

So they set up a rate limiter with Redis.



Rate Limiting API Requests

How it works:

- A counter is implemented using the Redis hash data structure
- The counter represents the number of requests allowed on each API endpoir period
- The counter gets decremented with each request
- Requests get rejected if the counter becomes 0
- The counter gets reset to its initial value after a specific period

While they deployed their app into production - it worked like a charm.

🔥 Bonus

Here are some more Redis use cases:

- Sorted sets can be used to create a leaderboard
- HyperLogLog can be used to estimate the number of items in a set
- Pub-Sub can be used to implement real-time messaging
- The geospatial index can be used to find nearby points within a radius
- Time series data type can be used for data analytics
- The list data type can be used to create a social network timeline
- The RedisSearch module supports full-text search and SQL-like query functic It's useful if the search index fits in memory or changes often
- The RedisJSON module can be used to store nested JSON data. It avoids de-serialization costs

The bottom line: Redis is more than just a cache. It runs in memory, thus provid faster reads and writes.

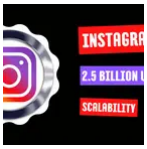
Also it can survive a crash with disk persistence.

Besides it supports atomic operations using transactions.

It's single-threaded, so there is no need for locks. Thus offering high performanc

Follow me on LinkedIn | YouTube | Threads | Twitter | Instagram

Thank you for supporting this newsletter. Consider sharing this post with your fri and get rewards. Y'all are the best.



How Instagram Scaled to 2.5 Billion Users

NEO KIM • 11 JUNE 2024

Read full story →

How YouTube Was Able to Support 2.49 Billion Users With My

NEO KIM • 31 MAY 2024

Read full story →

References

- 5 Redis Use Cases with Gur Dotan - Redis Labs
- Redis Use Case Examples for Developers
- What is Redis Cache?
- Redis Streams Explained
- RedisJSON Explained
- Redis Data Structures for Non-Redis Users
- Cache vs. Session Store
- Top 5 Caching Patterns
- Leaderboard System Design
- This Is How Stripe Does Rate Limiting to Build Scalable APIs
- Dave Nielsen: Top 5 uses of Redis as a Database | PyData Seattle 2015
- AWS re: Invent 2020: Beyond caching: Advanced design patterns in Redis
- RedisTimeSeries Explained
- Querying, Indexing, and Full-text Search in Redis
- Redis Lists Explained
- Redis Geospatial Explained
- Making Session Stores More Intelligent with Microservices
- Redis HyperLogLog Explained
- Redis Adopts Dual Source-Available Licensing - Hacker News

1
- Garnet
- Valkey
- KeyDB
- Redict

322 Likes · 33 Restacks

← Previous Nex

Discussion about this post

Comments Restacks

Write a comment...

cch 22 Jun 2024

For the "2. Queueing" use case, when there is a cache miss, since the processing of the reque decoupled, does queue worker simply update the cache after it gets data back from the db? V user get in return when there is a cache miss ?

LIKE (1) REPLY

Chips Ahoy Capital 28 Jun 2024

This is great - out of curiosity what app do you use to make your charts?

LIKE REPLY

1 reply

1 more comment...